

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 5
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Обход графа в ширину»

Выполнил
студент группы 24ВВВ2:

Воеводин И.А.
Гуляевский Д.С.
Каташов К.А.

Приняли
Юрова О.В.
Деев М.В.

Пенза, 2025

Цель работы

Обход графа в ширину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран
2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.
3. *Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2

1. *Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной.
2. *Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс **queue** и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Описание метода решения

Метод решения основан на алгоритме обхода графа в ширину (BFS) с использованием различных комбинаций структур данных.

На первом этапе создается матрица смежности графа. Пользователь вводит количество вершин, после чего программа динамически выделяет память для квадратной матрицы. Матрица заполняется случайными значениями: для каждой пары вершин i и j , кроме случаев когда i равно j , генерируется случайное число 0 или 1, определяющее наличие или отсутствие ребра между этими вершинами. Поскольку граф неориентированный, матрица делается симметричной - значение для пары (i,j) всегда равно значению для пары (j,i) . На главной диагонали

матрицы устанавливаются нули, что исключает наличие петель. После генерации матрица выводится на экран для визуализации структуры графа.

На втором этапе выполняется обход графа в ширину. Пользователь выбирает начальную вершину, с которой начинается обход. Алгоритм использует очередь для хранения вершин, которые нужно обработать. Начальная вершина помечается как посещенная и помещается в очередь. Затем в цикле из очереди извлекается очередная вершина, она выводится на экран как часть порядка обхода, и все смежные с ней непосещенные вершины добавляются в конец очереди и помечаются как посещенные. Этот процесс продолжается до тех пор, пока очередь не станет пустой, что означает, что все достижимые из начальной вершины узлы были обработаны. В результате на экран выводится последовательность вершин в порядке их посещения при обходе в ширину.

Листинг

Задание 1

```
#include <iostream>
#include <locale.h>
#include <time.h>
#include <cstdlib>
#include <queue>
using namespace std;

void BFS(int** G, int* vis, int numG, int s) {
    queue<int> q;
    vis[s] = 1;
    q.push(s);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        printf("%3d", v);
        for (int i = 0; i < numG; i++) {
            if (vis[i] == 0 && G[v][i] == 1) {
                q.push(i);
                vis[i] = 1;
            }
        }
    }
}

int main() {
    srand(time(NULL));
    setlocale(LC_ALL, "rus");

    int** G;
    int* vis;
    int numG, s;

    std::cout << "Введите кол-во элементов: " << std::endl;
    std::cin >> numG;

    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    vis = (int*)malloc(numG * sizeof(int));

    for (int i = 0; i < numG; i++) {
        vis[i] = 0;
        for (int j = 0; j < numG; j++) {
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d ", G[i][j]);
        }
        printf("\n");
    }
    printf("Введите нач вершину: ");
    scanf_s("%d", &s);
    printf("Порядок обхода: ");
    BFS(G, vis, numG, s);
    return 0;
}
```

Псевдокод

1-е задание

НАЧАЛО

Установить русскую локаль

Инициализировать генератор случайных чисел

ВВЕСТИ numG // количество вершин графа

// Выделение памяти для матрицы смежности и массива посещений

G = двумерный массив размером [numG][numG]

vis = массив размером [numG], заполненный нулями

// Генерация матрицы смежности

ДЛЯ i = 0 ДО numG-1

 ДЛЯ j = 0 ДО numG-1

 ЕСЛИ i == j ТО

 G[i][j] = 0 // нет петель

 ИНАЧЕ

 G[i][j] = случайное(0 или 1)

 G[j][i] = G[i][j] // симметричность

 КОНЕЦ ЕСЛИ

 КОНЕЦ ДЛЯ

КОНЕЦ ДЛЯ

// Вывод матрицы смежности

ВЫВЕСТИ "Матрица смежности:"

ДЛЯ i = 0 ДО numG-1

 ДЛЯ j = 0 ДО numG-1

 ВЫВЕСТИ G[i][j]

 КОНЕЦ ДЛЯ

 ПЕРЕВЕСТИ СТРОКУ

КОНЕЦ ДЛЯ

ВВЕСТИ s // начальная вершина для обхода

ВЫВЕСТИ "Порядок обхода:"

// Алгоритм BFS

СОЗДАТЬ очередь q

vis[s] = 1 // пометить начальную вершину как посещенную

q.добавить(s)

ПОКА q не пуста

 v = q.взять_первый() // извлечь вершину из начала очереди

 ВЫВЕСТИ v

// Обработка смежных вершин

ДЛЯ i = 0 ДО numG-1

 ЕСЛИ vis[i] == 0 И G[v][i] == 1 ТО

 vis[i] = 1

 q.добавить(i)

 КОНЕЦ ЕСЛИ

КОНЕЦ ДЛЯ

КОНЕЦ ПОКА

Освободить память
КОНЕЦ
2-е задание

Результаты работы программы

1-ое задание

```
Консоль отладки Microsoft Visual Studio
Введите кол-во элементов:
5
0 1 1 1 0
1 0 1 1 1
1 1 0 0 0
1 1 0 0 1
0 1 0 1 0
Введите нач вершину: 0
Порядок обхода: 0 1 2 3 4
C:\Users\Admin-PC\Desktop\Lab8\lab8\x64\Debug\lab8.exe (процесс 5868) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

Вывод

В ходе выполнения лабораторной работы были успешно реализованы алгоритмы обхода графа в ширину с использованием различных структур данных. Основной задачей являлось освоение принципов работы с графами и алгоритмом BFS на практике.

Была разработана программа на языке C++, которая генерирует матрицу смежности для неориентированного графа заданного размера. Программа корректно формирует симметричную матрицу, исключая петли и случайным образом устанавливая связи между вершинами. Реализованный алгоритм обхода в ширину демонстрирует правильную работу: начинается с выбранной пользователем вершины, последовательно посещает все достижимые вершины, используя очередь для управления порядком обхода.

Алгоритм BFS показал свою эффективность для обработки связных компонент графа. Использование стандартного класса `queue` из библиотеки STL обеспечило надежную работу с данными и упростило реализацию алгоритма. Программа наглядно отображает матрицу смежности и порядок обхода вершин, что позволяет легко проверить корректность ее работы.

В процессе работы были освоены ключевые аспекты теории графов: представление графов в виде матрицы смежности, принципы обхода в ширину, работа с динамическими структурами данных. Полученные знания могут быть применены для решения более сложных задач на графах, таких как поиск кратчайшего пути или анализ связности.